REPUBLIQUE DU BENIN (RB)

MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE (MESRS)

****

UNIVERSITE DE PARAKOU (UP)

****

INSTITUT UNIVERSITAIRE DE TECHNOLOGIE (IUT)

Année : 2025 – 2026

Cours : Programmation Python — Niveau L2

Approche : Apprentissage Par Projet (APP)

CAHIER DES CHARGES

Projet 9 — Agrégateur de News Personnalisé (Scraping)

Date de rédaction : 20 avril 2026

Présenté par :

HOUETO Jasmes

ADEBIYI Nicodème

Sous la supervision de :

Dr MOUSSE Mikael Ange

1. Présentation du Projet

1.1 Contexte

Dans le cadre du cours de Programmation Python (Niveau L2), chaque équipe de deux étudiants doit réaliser un projet logiciel complet en suivant une démarche par blocs techniques. Le présent document constitue le cahier des charges du Projet 9, présenté par Jasmes et Nicodème.

1.2 Description générale

Le projet consiste à développer un Agrégateur de News Personnalisé. Il s'agit d'un outil Python capable de collecter automatiquement des titres d'articles publiés sur plusieurs sites de presse en ligne, en fonction de mots-clés définis par l'utilisateur, et de les présenter de manière claire et structurée.

1.3 Objectifs pédagogiques

- Maîtriser le web scraping avec les bibliothèques requests et BeautifulSoup.
- Appliquer les principes de la Programmation Orientée Objet (POO).
- Utiliser les expressions régulières (Regex) pour le traitement de texte.
- Assurer la persistance des données (JSON ou CSV).
- Rédiger des tests unitaires avec Pytest.
- Travailler avec Git en équipe (GitHub ou GitLab).

2. Besoins Fonctionnels

Le tableau ci-dessous recense toutes les fonctionnalités que l'application doit offrir à l'utilisateur :

ID	Fonctionnalité	Priorité
F01	L'utilisateur saisit un ou plusieurs mots-clés de recherche.	Haute
F02	L'application scrape automatiquement plusieurs sites de presse.	Haute

F03	Les articles sont filtrés selon les mots-clés saisis.	Haute
F04	Les résultats affichent le titre, la source et le lien de chaque article.	Haute
F05	Les résultats peuvent être sauvegardés dans un fichier JSON ou CSV.	Moyenne
F06	L'utilisateur peut relancer une nouvelle recherche sans redémarrer le programme.	Moyenne
F07	L'application gère les erreurs réseau (site indisponible, timeout).	Haute
F08	L'utilisateur peut consulter l'historique des recherches précédentes.	Basse

3. Besoins Non Fonctionnels

3.1 Qualité du code

- Le code doit respecter la norme **PEP 8** (style officiel Python).
- Chaque module doit être documenté avec des docstrings.
- La gestion des exceptions doit être systématique.

3.2 Versioning

- Le projet doit être versionné avec Git (GitHub ou GitLab).
- Des commits réguliers et explicites sont obligatoires.
- Un fichier README.md doit décrire le projet et son installation.

3.3 Tests

- Les tests unitaires sont écrits avec Pytest.
- Un minimum de 5 tests unitaires couvrant les classes principales est attendu.

3.4 Performance

- Le scraping ne doit pas bloquer indéfiniment (timeout configuré).

- Les requêtes HTTP doivent être espacées pour éviter le bannissement IP.

4. Architecture Technique (POO)

Le projet est structuré autour de quatre classes principales :

Classe	Rôle	Attributs principaux
Article	Représente un article collecté.	titre, lien, source, date
Scraper	Se connecte à un site et extrait les titres d'articles.	url, nom_source, timeout
Aggregator	Coordonne plusieurs Scrapers et filtre par mots-clés.	scrapers, mots_cles, articles
Storage	Sauvegarde et charge les résultats en JSON ou CSV.	chemin_fichier, format

4.1 Structure des fichiers du projet

Fichier / Dossier	Description
scraper/article.py	Définition de la classe Article
scraper/scraper.py	Définition de la classe Scraper
scraper/aggregator.py	Définition de la classe Aggregator
scraper/storage.py	Définition de la classe Storage
data/	Dossier de sauvegarde des résultats (JSON/CSV)
tests/	Tests unitaires Pytest
main.py	Point d'entrée du programme
requirements.txt	Liste des dépendances Python
.gitignore	Fichiers exclus du versioning Git

README.md	Documentation du projet

5. Sites Cibles et Bibliothèques

5.1 Sites de presse envisagés

- Le Monde — https://www.lemonde.fr
- Le Figaro — https://www.lefigaro.fr
- BBC News — https://www.bbc.com/news

Note : Les sites cibles peuvent être ajustés selon leur accessibilité (certains sites bloquent le scraping). Des flux RSS pourront être utilisés en alternative.

5.2 Bibliothèques Python requises

Bibliothèque	Version	Rôle
requests	>=2.28	Effectuer les requêtes HTTP
beautifulsoup4	>=4.11	Parser le HTML et extraire les données
pytest	>=7.0	Écrire et exécuter les tests unitaires
lxml	>=4.9	Parser HTML/XML plus rapide (optionnel)

6. Découpage en Blocs et Planning

Le projet est découpé en 6 blocs techniques à valider successivement :

Bloc	Contenu	Échéance
Bloc 1	Cahier des charges, spécifications techniques, création du dépôt Git.	20/04/2026
Bloc 2	Environnement virtuel, Git opérationnel, types de données complexes.	22/04/2026

Bloc 3	Architecture POO : classes Article, Scraper, Aggregator, Storage.	22/04/2026
Bloc 4	Persistance des données (JSON ou CSV).	À définir
Bloc 5	Gestion des exceptions, tests unitaires avec Pytest.	À définir
Bloc 6	Interface utilisateur (CLI soignée ou GUI avec Tkinter/Flask).	À définir

7. Critères d'Évaluation

L'évaluation portera sur les points suivants :

Critère	Détail	Pondération
Fonctionnalité	Le scraping fonctionne et les résultats sont affichés correctement.	30%
Qualité POO	Les classes sont bien conçues, cohérentes et réutilisables.	25%
Qualité du code	Respect PEP 8, docstrings, lisibilité générale.	15%
Tests	Couverture des tests unitaires avec Pytest.	15%
Git	Historique de commits régulier et messages explicites.	10%
Soutenance	Présentation orale et démonstration du projet.	5%

8. Signatures

Rôle	Nom	Date	Signature
Étudiant 1	Jasmes	20/04/2026	

Étudiant 2	Nicodème	20/04/2026	
Enseignant(e)	A	20/04/2026	